

Projektdokumentation

Entwicklung eines eigenständigen WYSIWYG-Texteditors für das Modul „Mini-Kurs“ der Anwendung „GPB Verwaltungstools“

Abschlussprüfung Teil 2 – Fachinformatiker/-in Anwendungsentwicklung, 2027

Prüfungsteilnehmer

Name	Emanuel Kronas
Adresse	Wildenbruchstraße 46 B, 12435 Berlin
E-Mail	e.kronas@kronisoft.net
Ausbildungsberuf	Fachinformatiker für Anwendungsentwicklung
Azubi-Nummer	0003713294
Prüfungsausschuss	[TODO]

Ausbildungsbetrieb

Firma	Gesellschaft für Personalentwicklung und Bildung (GPB mbH)
Adresse	Juliusstr. 2, 12051 Berlin
E-Mail	info@gpb.de

Praktikumsbetrieb

Firma	Webware nach Maß
Adresse	Leuthener Str. 17, 10829 Berlin
Ansprechpartner	Rémy Mouton
E-Mail	kontakt@webware-nach-mass.de

Inhaltsverzeichnis

1 Einleitung.....	3
1.1 Projektumfeld.....	3
1.2 Problemanalyse.....	3
1.3 Projektschnittstellen.....	3
2 Projektdefinition.....	3
2.1 IST-Analyse.....	3
2.2 SOLL-Konzept.....	3
2.3 Projektziele.....	4
2.4 Projektabgrenzung.....	4
3 Projektplanung.....	4
3.1 Vorgehensmodell.....	4
3.2 Projektstrukturplan.....	5
3.3 Zeitplan.....	5
3.4 Gantt-Diagramm.....	6
3.5 Kostenplan.....	6
3.6 Qualitätssichernde Maßnahmen.....	7
3.7 Entwürfe.....	7
4 Durchführung.....	10
4.1 Implementierung.....	10
4.2 Qualitätskontrolle.....	11
4.3 Deployment.....	11
4.4 Außerplanmäßige Änderungen.....	11
5 Abschlussphase.....	13
5.1 Projektübergabe.....	13
5.2 Fachlicher Soll-Ist-Vergleich.....	13
5.3 Zeitlicher Soll-Ist-Vergleich.....	13
5.4 Fazit.....	13
6 Anhang.....	15
I. Anforderungen.....	15
II. Weitere Anhänge.....	15
7 Quellen.....	16

Abbildungsverzeichnis

Abbildung 1: Projektstrukturplan.....	5
Abbildung 2: ERD.....	8
Abbildung 3: GUI-Screenshot (Dozentenansicht, Testdaten).....	8
Abbildung 4: Use-Case-Diagramm.....	9

1 Einleitung

1.1 Projektumfeld

Dieses Projekt wurde im Rahmen des Pflichtpraktikums der Ausbildung zum Fachinformatiker für Anwendungsentwicklung in der Firma Webware nach Maß in Berlin durchgeführt. Die Firma entwickelt und betreut u. a. für ein Bildungsinstitut eine PHP-Webanwendung mit Bereichen für Leitung, Verwaltung, Dozenten und Teilnehmer.

Betreut wurde das Projekt von Rémy Mouton.

1.2 Problemanalyse

Innerhalb des Moduls „Mini-Kurs“ pflegen Dozenten Kursinhalte in Textfeldern mit Formatierungsmöglichkeiten (fett, kursiv, unterstrichen, Farben, Listen, Bildeinbindung). Zur Textformatierung wird bislang wahlweise eine einfache Tag-Button-Lösung (die rohe HTML-Tags in eine Textarea einfügt, ohne echte visuelle Vorschau) oder der kommerzielle Cloud-Editor TinyMCE (Lizenz über tiny.cloud, API-Key an die Domain kronisoft.net gebunden) eingesetzt.

Beide bestehenden Lösungen weisen Nachteile auf: Die Tag-Button-Lösung zeigt keine echte WYSIWYG-Vorschau, TinyMCE bringt eine Abhängigkeit von einem externen, potenziell kostenpflichtigen Anbieter mit sich (Lizenzbindung an die Domain, eingeschränkte Kontrolle über Ladeverhalten und Datenschutz durch externes Nachladen von Skripten). Zusätzlich wird der eingegebene HTML-Inhalt bislang ohne serverseitige Prüfung gespeichert, wodurch ein Cross-Site-Scripting-Risiko (XSS) besteht.

1.3 Projektschnittstellen

- Einbindung im Betrieb: Erweiterung des bestehenden Moduls „Mini-Kurs“ innerhalb der produktiven Anwendung des Betriebs (kein eigenständiges Projekt von Grund auf)
- Ansprechpartner: Rémy Mouton (Betreuer, Webware nach Maß)
- Technologien/Tools zur Anbindung: PHP, MySQL, JavaScript, HTML/CSS; Versionsverwaltung über Git/GitHub (Repository gpb_praktikum); Deployment per FileZilla-FTP auf Plesk-Hosting (kronisoft.net)

2 Projektdefinition

2.1 IST-Analyse

Aktuell existieren zwei parallele, unabhängig voneinander nutzbare Formatierungswege im Mini-Kurs-Formular: eine Textarea mit Tag-Button-Symbolleiste (fügt rohe HTML-Tags an der Cursorposition ein) und optional der TinyMCE-Cloud-Editor, zwischen denen der Dozent per Checkbox wechseln kann (Präferenz wird im Browser via localStorage gemerkt). Der gespeicherte HTML-Inhalt landet unverändert, ohne serverseitige Prüfung, in der Tabelle `gpb_mini_paragraph`. Bildanhänge werden über eine separate Anhangsverwaltung (Tabelle `gpb_mini_anhang`, Endpunkt `mini_anhang_download.php`) hochgeladen und verwaltet.

2.2 SOLL-Konzept

Aus Anwendersicht (Dozent) soll ein eigenständiger WYSIWYG-Editor entstehen, der TinyMCE vollständig ersetzt und dabei:

- eine echte visuelle Vorschau der Formatierung während der Eingabe anbietet,
- die bestehende Funktionalität abdeckt (Fett, Kursiv, Unterstrichen, Textfarbe, Listen, Bildanhang einfügen),
- kompatibel zur bestehenden Anhangsverwaltung bleibt,
- eine umschaltbare Quellcode-Ansicht bereitstellt,
- ohne kostenpflichtige Fremdbibliotheken auskommt.

2.3 Projektziele

Aus Sicht des Programmierers umfasst das Projekt zusätzlich die für den Anwender nicht sichtbaren Teile:

- Frontend-Editor auf Basis von contenteditable und der Selection-/Range-API des Browsers, ohne das veraltete document.execCommand
- Serverseitige Whitelist-Sanitisierung (MiniHtmlSanitizer.php) des gespeicherten HTML auf DOMDocument-Basis, um XSS-Risiken zu minimieren
- Nahtlose Integration in die bestehende Speicherlogik (Tabelle gpb_mini_paragraph, Aktionen hinzufuegen/speichern in dozent/mini_kurs_sehen.php)
- Ablösung der TinyMCE-CDN-Einbindung und des zugehörigen API-Keys

2.4 Projektabgrenzung

Folgende Arbeiten sind nicht Teil des Projektes:

- Systemweite Ausrollung des Editors auf weitere Formulare außerhalb des Mini-Kurs-Bereichs
- Neuentwicklung des Datei-Upload-Mechanismus für Anhänge (wird unverändert weiterverwendet)
- Vollständige Nachbildung aller Funktionen kommerzieller Editoren (z. B. Tabellen, Spalten-Layouts)
- Umsetzung des Kann-Kriteriums Undo/Redo

3 Projektplanung

3.1 Vorgehensmodell

Das Projekt wurde nach einem klassischen Phasenmodell (wasserfallartig) mit sechs aufeinanderfolgenden Phasen durchgeführt: Analyse & Feinkonzept, Editor-Grundgerüst, Bild-/Anhang-Integration, Serverseitige Sanitisierung, Testing und Dokumentation. Ergänzend erfolgte eine regelmäßige Abstimmung mit dem Betreuer Rémy Mouton, u. a. zur Anpassung von Projektbezeichnung und Formulierungen im Projektantrag.

3.2 Projektstrukturplan

Für die Erstellung des Projektstrukturplans wurde ein selbst programmiertes Diagramm-Tool eingesetzt (<https://r-mouton.gpbmedien.de/diagramm/>); die Ergebnisse sind funktional, aber gestalterisch noch ausbaufähig. Die hier gewählte, eher senkrechte Darstellung folgt einer traditionellen Notation; die offizielle DIN-Darstellung eines PSP verwendet stattdessen eckige Verbindungslinien (vgl. <https://de.wikipedia.org/wiki/Projektstrukturplan>).

Der vollständige Projektstrukturplan ist im Anhang abgebildet (Abschnitt III).

3.3 Zeitplan

Der detaillierte Zeitplan mit Teilprozessen ist in Tabelle 1 dargestellt.

Prozess	Teilprozess	Stunden
Definitionsphase (Phase 1) – 8 Std.	Ist-Analyse der bestehenden Tag-Button-/TinyMCE-Lösung	3
	Anforderungsdefinition (Muss-/Kann-Kriterien)	3
	Festlegung der Zielarchitektur	2
Editor-Grundgerüst (Phase 2) – 20 Std.	HTML-Grundgerüst mit contenteditable-Fläche und Toolbar-Markup	4
	Aufbau Selection-/Range-Zugriff	4
	Formatierungsfunktionen Fett/Kursiv/Unterstrichen/Farbe inkl. Toggle-Logik	6

Prozess	Teilprozess	Stunden
	Listenfunktion (geordnet/ungeordnet)	3
	Umschaltbare Quellcode-Ansicht	3
Bild-/Anhang-Integration (Phase 3) – 12 Std.	Cursorposition vor Upload zwischenspeichern (Range-Objekt)	3
	Anbindung an mini_anhang_download.php	5
	Bild-Einfügen an Cursorposition	4
Serverseitige Sanitisierung (Phase 4) – 12 Std.	Whitelist-Konzept festlegen	3
	Sanitisierungsfunktion implementieren (MiniHtmlSanitizer.php)	6
	Integration in den Speicherprozess (dozent/mini_kurs_sehen.php)	3
Testing (Phase 5) – 12 Std.	Funktionstests F1–F13 (Editor selbst)	3
	Browstertests Desktop (Chrome, Firefox, Edge)	3
	Browstertests mobile/responsive Ansicht	2
	XSS-Testfälle gegen Sanitisierung	4
Dokumentation (Phase 6) – 16 Std.	Testprotokolle abschließen	3
	Projektdokumentation erstellen	10
	Screenshots/Anhang zusammenstellen	3
Gesamtstunden		80

Tabelle 1: Zeitplan detailliert

3.4 Gantt-Diagramm

Die Verteilung der sechs Phasen auf die zehn Arbeitstage des Projektzeitraums ist im Anhang tabellarisch dargestellt (Abschnitt III).

3.5 Kostenplan

Da es sich um ein internes Ausbildungsprojekt ohne Rechnungsstellung an einen externen Kunden handelt, sind die folgenden Stundensätze intern abgestimmte Richtwerte.

Personalkosten

Personal	Stundenlohn	Stundenanzahl	Gesamtlohn
Eigener Lohn (Azubi)	6,94 €/h	80	555,20 €
Betreuung Rémy Mouton	22,00 €/h	~4	88,00 €

Fixkosten Pauschal

Kostenart	Kosten/h	Stunden	Gesamtkosten
Arbeitsplatz (PC, Strom)	6,00 €/h	80	480,00 €
Hosting-Anteil (Plesk)	15,00 € (Pauschale)	–	15,00 €
Projektgesamtkosten			1.138,20 €

Tabelle 3: Kostenplan

3.6 Qualitätssichernde Maßnahmen

Konstruktive Maßnahmen	Analytische Maßnahmen
Detaillierte Anforderungsliste, siehe Anhang I	Entwicklertests (jsdom-Tests für Editor-Logik, siehe CHANGELOG 2026-08)
Detaillierte Planung (Zeitplan, Projektstrukturplan)	White-Box-Test / Code Review durch Betreuer Rémy Mouton (Feedback-Runde 2026-07-17, siehe CHANGELOG)
Einhaltung bestehender Namens-/Coding-Konventionen des Projekts (deutsche Funktionsnamen, z. B. gpbMiniHtmlSaeubern())	Black-Box-Tests: 34 Testfälle (Funktions-, Browser- und XSS-Tests), siehe Testprotokoll (Anhang / docs/Testprotokoll_WYSIWYG_Editor.md)
Selektives Staging (git add [Dateiname] statt git add .), um sensible Dateien wie db.php nicht zu committen	Manuelle Browsertests: Chrome, Firefox, Edge (Desktop und mobile/responsive Ansicht)
Entfernen von Debug-Ausgaben (display_errors, error_reporting) vor jedem Commit	XSS-Testfälle gegen die Sanitisierungsfunktion (15 Testfälle, siehe Testprotokoll Abschnitt 4)
Pflege des CHANGELOG.md mit datierten Einträgen	Keine dedizierten automatisierten Unit-Tests für PHP-Backend (manuell per CLI getestet)
Whitelist-Prinzip (statt Blacklist) bei der serverseitigen Sanitisierung	

Tabelle 4: Qualitätssicherungsplan

3.7 Entwürfe

3.7.1 Datenbank

Die Kursinhalte werden in der bestehenden Tabelle `gpb_mini_paragraph` gespeichert (`id`, `kursid`, `nummer`, `titel`, `inhalt`). Bildanhänge liegen in einer separaten Tabelle `gpb_mini_anhang` mit Fremdschlüssel `paragraphid`, die auf `gpb_mini_paragraph.id` verweist (1:n-Beziehung: ein Paragraph kann mehrere Anhänge haben). Für dieses Projekt wurde keine neue Tabelle angelegt; lediglich ein historischer Schemafehler wurde behoben: Die Spalte `id` hatte kein `AUTO_INCREMENT/PRIMARY KEY`, wodurch neue Paragraphen stets `id=0` erhielten – behoben per `ALTER TABLE gpb_mini_paragraph MODIFY id INT(11) NOT NULL AUTO_INCREMENT, ADD PRIMARY KEY (id)`.

Das vollständige ER-Diagramm ist im Anhang abgebildet (Abschnitt III).

3.7.2 Graphical User Interface

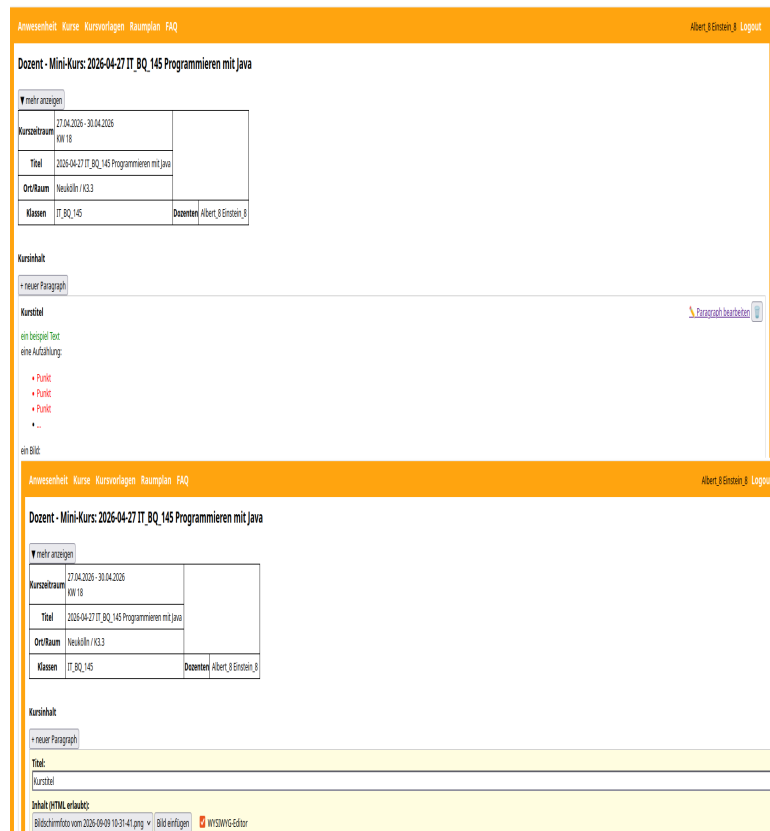


Abbildung 1: GUI-Screenshot (Dozentenansicht, Testdaten)

3.7.3 UML-Diagramme

Zentraler Anwendungsfall: Ein Dozent formatiert Text im Editor (Fett/Kursiv/Unterstrichen/Farbe/Liste), fügt optional einen Bildanhang ein, wechselt bei Bedarf in die Quellcode-Ansicht und speichert den Paragraphen; die serverseitige Sanitisierung läuft dabei transparent im Hintergrund.

Das vollständige Use-Case-Diagramm ist im Anhang abgebildet (Abschnitt III).

4 Durchführung

4.1 Implementierung

4.1.1 Software-Architektur

Die Anwendung folgt einer klassischen Client-Server-Architektur mit rollenspezifischer Ordnerstruktur (dozent/, tn/, verwaltung/, leitung/). Der neue Editor ist strikt in Frontend und Backend getrennt: Im Frontend kapselt das JavaScript-Objekt MiniWysiwyg (mini_wysiwyg.js) sämtliche Editor-Logik (Formatierung, Toggle-Verhalten, Quellcode-Umschaltung) und wird über init()/destroy()/getContent()/insertContent() angesprochen – als direkter Drop-in-Ersatz für die bisherigen tinymce.init()/get()/remove()-Aufrufe. Im Backend übernimmt die eigenständige Funktion gpbMiniHtmlSaeubern() (MiniHtmlSanitizer.php) die serverseitige Bereinigung nach dem Whitelist-Prinzip, unabhängig davon, was das Frontend liefert (Defense-in-Depth: Client formatiert, Server validiert unabhängig davon erneut).

4.1.2 Funktion „Formatierung“

Fett, Kursiv, Unterstrichen und Textfarbe laufen über dieselbe generische Funktion toggleInline(editor, tagName, styleProp, styleValue). Sie prüft zunächst per closestWithinEditor(), ob die aktuelle Selektion bereits vollständig von einem passenden Element (gleicher Tag, ggf. gleicher Style) umschlossen ist: Ist das der Fall, wird das Element entpackt (Toggle AUS). Andernfalls wird ein neues Inline-Element um die Selektion gelegt (Toggle AN). Ein bekannter Sonderfall von range.extractContents() – bei exakter Deckung von Selektion und Tag bleibt sonst eine leere Hülle zurück – wird dabei über eine gesonderte Fallunterscheidung im Code behandelt.

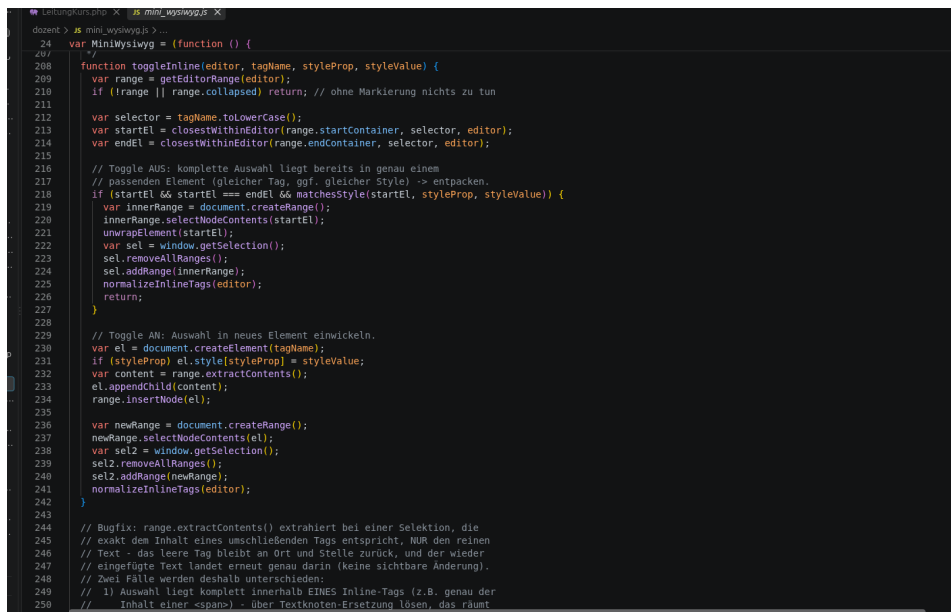
The image shows a code editor with the file mini_wysiwyg.js open. The code defines a function toggleInline(editor, tagName, styleProp, styleValue). It first checks if the selection is already within an element of the same tag and style. If so, it removes the element (Toggle AUS). If not, it creates a new element, wraps the selection in it, and inserts it (Toggle AN). The code also includes a bugfix for range.extractContents() which might return an empty string if the selection exactly matches the tag, and a comment about handling nested inline tags like .</div>

Abbildung 2: Funktion toggleInline() (mini_wysiwyg.js)

4.1.3 Funktion „Anhang platzieren“

Nach dem Hochladen eines Bildanhangs über die bestehende Anhangsverwaltung wird das Bild nicht ans Dokumentenende, sondern exakt an der Cursorposition eingefügt, an der sich der Fokus vor dem Upload befand. Der Aufrufer merkt sich dazu die Range vor dem Öffnen des Upload-Dialogs; insertContent(textareaId, html) baut daraus über createContextualFragment() ein einfügbares Fragment und ersetzt den aktuell selektierten Bereich damit (range.deleteContents() + range.insertNode()). Liegt keine gültige Range mehr vor, fällt die Funktion auf das Editorende zurück.

```

dozent > ./mini_wysiwyg.js > ...
24 var MiniWysiwyg = (function () {
622 function destroy(textareaId) {
624   if (!instance) return;
625   if (instance.sourceTextarea) instance.editor.innerHTML = instance.sourceTextarea.value;
626   syncTextarea(instance);
627   instance.wrapper.remove();
628   instance.textarea.style.display = '';
629   delete instances[textareaId];
630 }
631
632 function getContent(textareaId) {
633   var instance = instances[textareaId];
634   if (!instance) return null;
635   var html = instance.sourceTextarea ? instance.sourceTextarea.value : instance.editor.innerHTML;
636   return stripTrailingEmptyParagraphs(html);
637 }
638
639 function insertContent(textareaId, html) {
640   var instance = instances[textareaId];
641   if (!instance) return;
642   var range = getEditorRange(instance.editor);
643   if (!range) {
644     instance.editor.focus();
645     range = document.createRange();
646     range.selectNodeContents(instance.editor);
647     range.collapse(false);
648   }
649   var frag = range.createContextualFragment(html);
650   range.deleteContents();
651   range.insertNode(frag);
652   syncTextarea(instance);
653 }
654
655 return {
656   init: init,
657   destroy: destroy,
658   getContent: getContent,
659   insertContent: insertContent
660 };
661 })();
662

```

Abbildung 3: Funktion *insertContent()* (*mini_wysiwyg.js*)

4.1.4 Implementierung der Klassen

In JavaScript und PHP wurden für dieses Projekt keine eigenen Klassen definiert. Der Editor (*mini_wysiwyg.js*) ist als Revealing-Module-Pattern umgesetzt: Eine IIFE kapselt den internen Zustand (*instances*) und exponiert über ein öffentliches Objekt (*init*, *destroy*, *getContent*, *insertContent*) eine API, ohne dass mehrere parallel aktive, unterschiedlich konfigurierte Instanzen benötigt würden. Im Backend ist die Sanitisierung als eigenständige, zustandslose Funktion *gpbMiniHtmlSaeubern()* realisiert statt als Klasse, da sie eine reine Eingabe-Ausgabe-Transformation ohne Objektzustand ist.

Eine objektorientierte Kapselung – etwa eine Editor-Klasse pro Instanz oder eine Sanitizer-Klasse mit konfigurierbarer Whitelist – wäre technisch möglich und ließe sich bei wachsendem Funktionsumfang sinnvoll nachrüsten. Für den aktuellen Funktionsumfang wurde dieser Mehraufwand als nicht angemessen eingestuft.

Kernstück der Sanitisierung ist die Funktion *gpbMiniHtmlSaeubern(string \$html)*: string in *MiniHtmlSanitizer.php*. Sie parst den HTML-Input über *DOMDocument*, durchläuft rekursiv alle Knoten (*gpbMiniKnotenSaeubern()*) und entfernt alle Tags/Attribute, die nicht auf der Whitelist stehen (*strong*, *em*, *u*, *span* mit ausschließlich *style="color:..."*, *ul*, *ol*, *li*, *p*, *br*, *img* mit ausschließlich *src/alt* aus dem eigenen Anhangspfad). Entfernte, aber nicht erlaubte Elternelemente werden aufgelöst, ihr (bereinigter) Inhalt bleibt erhalten ("Hochziehen"); *script*, *iframe*, *object*, *style*, *embed*, *link* und *meta* werden dagegen vollständig inklusive Inhalt entfernt.

```

1  MiniHtmlSanitizer.php
2  /**
3   * Serverseitige Sanitisierung für den Mini-WYSIWYG-Editor.
4   * Whitelist-Prinzip: nur explizit erlaubte Tags/Attribute bleiben erhalten,
5   * alles andere wird entfernt bzw. der Tag selbst entfernt (Inhalt ggf. promoted).
6   */
7
8  function gpbMiniHtmlSaeubern(string $html): string
9  {
10     $erlaubteTags = ['strong', 'em', 'u', 'span', 'ul', 'ol', 'li', 'p', 'br', 'img'];
11
12     $dom = new DOMDocument();
13     libxml_use_internal_errors(true);
14
15     // UTF-8 sicherstellen, in Wrapper-Div packen, damit wir einen definierten Root haben
16     $wrapped = '<?xml encoding="utf-8" ?><div>' . $html . '</div>';
17     $dom->loadHTML($wrapped, LIBXML_HTML_NOIMPLIED | LIBXML_HTML_NODEFDTD);
18     libxml_clear_errors();
19
20     $wurzel = $dom->getElementsByTagName('div')->item(0);
21     if ($wurzel === null) {
22         return '';
23     }
24
25     gpbMiniKnotenSaeubern($wurzel, $erlaubteTags);
26
27     $ergebnis = '';
28     foreach ($wurzel->childNodes as $kind) {
29         $ergebnis .= $dom->saveHTML($kind);
30     }
31
32     return $ergebnis;
33 }
34
35 function gpbMiniKnotenSaeubern(DOMNode $knoten, array $erlaubteTags): void
36 {
37     // Snapshot der Kinder, da wir die Liste während des Durchlaufs verändern
38     $kinder = iterator_to_array($knoten->childNodes);
39
40     foreach ($kinder as $kind) {
41         if ($kind->nodeType === XML_TEXT_NODE) {
42             continue;
43         }
44
45         if ($kind->nodeType !== XML_ELEMENT_NODE) {

```

Abbildung 4: Funktionsköpfe `gpbMiniHtmlSaeubern()` und `gpbMiniKnotenSaeubern()` (`MiniHtmlSanitizer.php`)

4.1.5 Implementierung der Datenbank

Es wurde keine neue Tabellenstruktur benötigt. Behoben wurde ein bestehender kritischer Fehler an der Tabelle `gpb_mini_paragraph` (fehlendes `AUTO_INCREMENT/PRIMARY KEY` auf der `id`-Spalte), der dazu führte, dass alle neu angelegten Paragraphen die `id 0` erhielten und sich dadurch gegenseitig überschrieben bzw. beim Löschen falsch zugeordnet wurden.

	Feld	Typ	Kollation	Attribute	Null	Standard	Extra
☐	id	int(11)			Nein	Kein	auto_increment
☐	kursid	int(11)			Nein	Kein	
☐	nummer	int(11)			Nein	0	
☐	titel	varchar(255)	latin1_swedish_ci		Nein		
☐	inhalt	text	latin1_swedish_ci		Nein	"	

Abbildung 5: Tabellenstruktur `gpb_mini_paragraph` nach der Korrektur (`id`: Primärschlüssel, `auto_increment`)

4.1.6 Schnittstellenanbindung – API

Für dieses Projekt wurde bewusst auf die Anbindung externer Bibliotheken oder APIs verzichtet – das ist ausdrücklich Teil des Projektziels, da genau die Abhängigkeit von der externen TinyMCE-Cloud-Bibliothek (`tiny.cloud`) durch den Editor ersetzt werden sollte. Die einzige bestehende Schnittstelle, die weiterverwendet wird, ist der interne Endpunkt `mini_anhang_download.php` zur Auslieferung von Bildanhängen.

4.2 Qualitätskontrolle

Die in 3.6 geplanten Maßnahmen wurden vollständig umgesetzt, mit Ausnahme dedizierter automatisierter Unit-Tests für das PHP-Backend – die serverseitige Sanitisierung wurde stattdessen manuell per PHP-CLI mit gezielten XSS-Testfällen geprüft (siehe 4.2.1).

4.2.1 Blackbox-Test

Insgesamt wurden 34 Testfälle in vier Kategorien durchgeführt (vollständiges Testprotokoll siehe Anhang /

docs/Testprotokoll_WYSIWYG_Editor.md); alle 34 wurden bestanden, ein während der Testphase gefundener Bug (unvollständige rekursive Bereinigung beim Hochziehen von Kindelementen, Testfall X9) wurde noch vor Abschluss der Tests behoben und erneut erfolgreich verifiziert.

Kategorie	Testfall-IDs	Anzahl	Bestanden
Funktionstests (Editor selbst)	F1–F13	13	13
Browsertests Desktop (Chrome, Firefox, Edge)	B1–B3	3	3
Browsertests mobile/responsive Ansicht	M1–M3	3	3
XSS-Testfälle gegen Sanitisierung	X1–X15	15	15
Gesamt		34	34

Tabelle 5: Testprotokoll (Zusammenfassung)

4.2.2 Whitebox-Test / Code-Review

Der Betreuer Rémy Mouton hat den Code während der gesamten Projektlaufzeit begleitet und regelmäßig Feedback gegeben, u. a. zu Formulierungen im Projektantrag und zur Zeitplanung. Ein dokumentiertes Beispiel für Betreuer-Feedback, das direkt zu Code-Änderungen führte, findet sich im CHANGELOG unter dem Eintrag vom 17.07.2026 (Zusammenlegung von Tabellenspalten, Anpassung der Zeilenumbruch-Normalisierung).

4.3 Deployment

Das Projekt wird in zwei getrennten Schritten ausgeliefert: git push aktualisiert ausschließlich das GitHub-Repository (Emu77/gpb_praktikum); der eigentliche Produktivserver (Plesk-Hosting, kronisoft.net) wird separat per FileZilla-FTP bestückt. Beide Schritte sind zwingend erforderlich – reines git push hat keine Auswirkung auf die Live-Umgebung.

4.4 Außerplanmäßige Änderungen

Während der Projektlaufzeit wurde ein sicherheitsrelevanter Vorfall entdeckt und behoben, der nicht Teil der ursprünglichen Planung war: Ein Ordner secret/ mit Klartext-Zugangsdaten (u. a. ein Passwort für einen Basic-Auth-Account) war über rund zwei Monate im öffentlichen GitHub-Repository einsehbar. Der Ordner wurde lokal gelöscht, mittels git filter-repo --path secret/ --invert-paths vollständig aus der Git-Historie entfernt (in zwei Durchgängen, da betroffene Dateien in älteren Commits sowohl unter secret/ als auch gpb/secret/ lagen) und per Force-Push überschrieben. Das betroffene Zugangsdatum wurde zusätzlich rotiert. .gitignore wurde entsprechend um secret/ sowie um *_alt/*_alt.* (lokale Backup-Dateien) ergänzt.

Zusätzlich wurden nach Abschluss der ursprünglich geplanten 80 Stunden zwei der in 2.4 abgegrenzten Kann-Kriterien mit geringem Aufwand nachgezogen: Tastenkombinationen (Strg+B/I/U) stellten sich bei der Durchsicht als bereits während Phase 2 mitimplementiert heraus (bindKeyboardShortcuts() in mini_wysiwyg.js) und mussten daher nicht mehr umgesetzt werden. Für das responsive Toolbar-Layout wurde ergänzend eine CSS-Media-Query (max-width: 480px) eingefügt, die auf schmalen Viewports größere Tap-Ziele und ein ausgeblendetes Text-Label beim Listen-Button vorsieht – als gezielte Reaktion auf die in Testfall M2 (320×480) dokumentierte Beobachtung enger Toolbar-Buttons. Beide Ergänzungen liegen außerhalb des ursprünglich budgetierten 80-Stunden-Rahmens; auf Undo/Redo wurde weiterhin bewusst verzichtet (siehe 5.4.1).

5 Abschlussphase

5.1 Projektübergabe

Die Übergabe an die Firma erfolgte über das bereits bestehende Git-Repository des Projekts auf GitHub. Sämtliche Änderungen wurden während der Projektlaufzeit in einzelnen, nachvollziehbaren Commits mit beschreibenden Commit-Messages übertragen; die vollständige Historie – inklusive des während der Testphase behobenen Bugs und der nachträglichen Kann-Kriterien – bleibt damit dauerhaft einsehbar und dient zugleich als technische Dokumentation für andere Entwickler im Team. Ergänzend fasst das CHANGELOG.md die wichtigsten Änderungen in chronologischer, gut lesbarer Form zusammen, ohne dass dafür der komplette Commit-Verlauf durchsucht werden muss. Eine gesonderte Programmierer-Dokumentation über die vorliegende Projektdokumentation hinaus war angesichts der überschaubaren Codebasis (eine JavaScript-Datei, eine PHP-Sanitisierungsfunktion, punktuelle Anpassungen an bestehenden Dateien) nicht erforderlich.

An den Betreuer Rémy Mouton wurde der fertige Editor direkt live im Dozentenbereich der produktiven Anwendung unter kronisoft.net vorgeführt und dort gemeinsam getestet – u. a. das Formatieren von Text, das Einfügen von Bildanhängen, der Wechsel in die Quellcode-Ansicht sowie die responsive Darstellung auf schmalen Bildschirmen. Rückmeldungen aus dieser Vorführung (etwa zur Formulierung von Projektantrag und Projektbezeichnung) flossen direkt in die Dokumentation ein. Eine separate Schulung der Dozenten als Endanwender war nicht erforderlich, da der neue Editor die bisherige Bedienung (Symbolleiste, Bild einfügen, Speichern) unverändert fortführt und keine neuen Arbeitsschritte einführt.

5.2 Fachlicher Soll-Ist-Vergleich

Anforderung	Erreicht	Beleg
Formatierung Fett/Kursiv/Unterstrichen/Farbe mit Toggle-Off	✓	F1–F7
Listenfunktion (geordnet/ungeordnet)	✓	F8
Bild-Anhang an Cursorposition einfügen	✓	F9
Umschaltbare Quellcode-Ansicht	✓	F11
Speichern als validiertes, bereinigtes HTML	✓	X1–X15
Wiederherstellung des zuletzt gespeicherten Zustands	✓	F10
Kann-Kriterium: Undo/Redo	✗ (nicht umgesetzt)	außerhalb Projektumfang
Kann-Kriterium: Tastenkombinationen	✓	<code>bindKeyboardShortcuts()</code> in <code>mini_wysiwyg.js</code>
Kann-Kriterium: Responsive Toolbar-Layout	✓ (nachträglich ergänzt)	Media Query in <code>mini_wysiwyg.css</code> , siehe 4.4

Tabelle 6: Fachlicher Soll-Ist-Vergleich

5.3 Zeitlicher Soll-Ist-Vergleich

SOLL-Werte aus dem Umsetzungs-Zeitplan; IST-Werte nachträglich rekonstruiert aus dem Projektverlauf (Chat-Verlauf und Git-Historie) – kein exaktes Zeiterfassungsprotokoll, sondern eine plausible Annäherung. Phase 2 (Editor-Grundgerüst) und Phase 4/5 (Sanitisierung/Testing) benötigten etwas mehr Zeit als vorgesehen – u. a. durch die gründlichere Absicherung der Range-Logik bei verschachtelten Selektionen sowie den in Testfall X9 gefundenen und behobenen Bug. Phase 3 (Bild-Integration) und Phase 6 (Dokumentation) liefen dafür etwas zügiger als geplant. In Summe wurde der budgetierte Rahmen von 80 Stunden eingehalten.

Phase	SOLL	IST
Phase 1 – Analyse & Feinkonzept	8	8
Phase 2 – Editor-Grundgerüst	20	21

Phase	SOLL	IST
Phase 3 – Bild-/Anhang-Integration	12	11
Phase 4 – Serverseitige Sanitisierung	12	13
Phase 5 – Testing	12	13
Phase 6 – Dokumentation	16	14
Gesamtstunden	80	80

Tabelle 7: Zeitlicher Soll-Ist-Vergleich

5.4 Fazit

5.4.1 Sachlicher Fazit

Der eigenständige WYSIWYG-Editor wurde erfolgreich entwickelt und ersetzt TinyMCE vollständig im Mini-Kurs-Bereich. Alle Muss-Kriterien wurden umgesetzt und durch 34 Testfälle abgesichert (0 offen, 0 fehlgeschlagen). Von den Kann-Kriterien wurden Tastenkombinationen und ein responsives Toolbar-Layout zusätzlich umgesetzt (siehe 4.4); auf Undo/Redo wurde bewusst verzichtet, da eine eigene History-Verwaltung angesichts der eigenen, nicht auf execCommand basierenden Formatierungslogik einen unverhältnismäßig hohen Zusatzaufwand kurz vor Projektabschluss bedeutet hätte.

5.4.2 Persönliches Fazit

Die Entwicklung des eigenen WYSIWYG-Editors war für mich der lehrreichste Teil des Projekts. Der Verzicht auf document.execCommand zugunsten der Selection- und Range-API bedeutete zunächst Mehraufwand, da ich viele Funktionen (Formatierung, Cursor-Positionierung, Undo/Redo-Verhalten) von Grund auf selbst nachbilden musste, die execCommand-basierte Editoren wie TinyMCE „kostenlos“ mitbringen. Gerade die korrekte Behandlung von Selektionen über mehrere DOM-Knoten hinweg sowie die Synchronisation zwischen sichtbarem Editor-Zustand und dem serverseitig sanitisierten HTML haben mir gezeigt, wie viel Komplexität moderne Editor-Bibliotheken üblicherweise verbergen.

Besonders wertvoll war die Erkenntnis, dass die bewusste Entscheidung gegen eine veraltete, aber weit verbreitete API (execCommand) sich langfristig auszahlt: Der Editor ist dadurch unabhängiger von Browser-spezifischen Eigenheiten und leichter wartbar. Der Zeitaufwand für Eigenimplementierung und die 34 Testfälle war höher als ursprünglich geplant, hat sich aber gelohnt.

Rückblickend würde ich die Selection-/Range-Logik von Anfang an modularer aufbauen, um einzelne Formatierungsfunktionen isolierter testen zu können. Insgesamt hat mir das Projekt gezeigt, dass sich der Mehraufwand einer sauberen Eigenlösung gegenüber einer schnellen, aber technisch überholten Lösung auszahlt.

6 Anhang

I. Anforderungen

Nr.	Anforderung
1	Das System muss dem Anwender eine echte visuelle Vorschau der Formatierung während der Eingabe anbieten.
2	Das System muss Formatierung (fett, kursiv, unterstrichen, Textfarbe) über eine Symbolleiste unterstützen, inkl. Toggle-Off bei erneutem Klick.
3	Das System muss Listenformatierung (geordnet/ungeordnet) unterstützen.
4	Das System muss das Einfügen von Bildanhängen aus der bestehenden Anhangsverwaltung an der Cursorposition ermöglichen.
5	Das System muss eine umschaltbare Quellcode-Ansicht (HTML) neben der WYSIWYG-Ansicht bieten.
6	Das System muss den Inhalt beim Speichern serverseitig validieren und bereinigen (Sanitisierung).
7	Das System muss den zuletzt gespeicherten Zustand beim erneuten Laden wiederherstellen.
8	Das System sollte ohne kostenpflichtige Fremdbibliotheken (z. B. TinyMCE-Lizenz) auskommen.
9	Das System kann eine Undo/Redo-Funktion anbieten.
10	Das System kann Tastenkombinationen (z. B. Strg+B für Fett) unterstützen.
11	Das System kann eine responsive Darstellung für mobile Endgeräte bieten.
12	Das System muss nicht alle Funktionen kommerzieller Editoren (z. B. Tabellen, Spalten-Layouts) nachbilden.

Tabelle 8: Anforderungen

II. Weitere Anhänge

- Testprotokoll_WYSIWYG_Editor.md (vollständiges Testprotokoll, 34 Testfälle)
- CHANGELOG.md (Auszug August 2026)

III. Diagramme und Tabellen

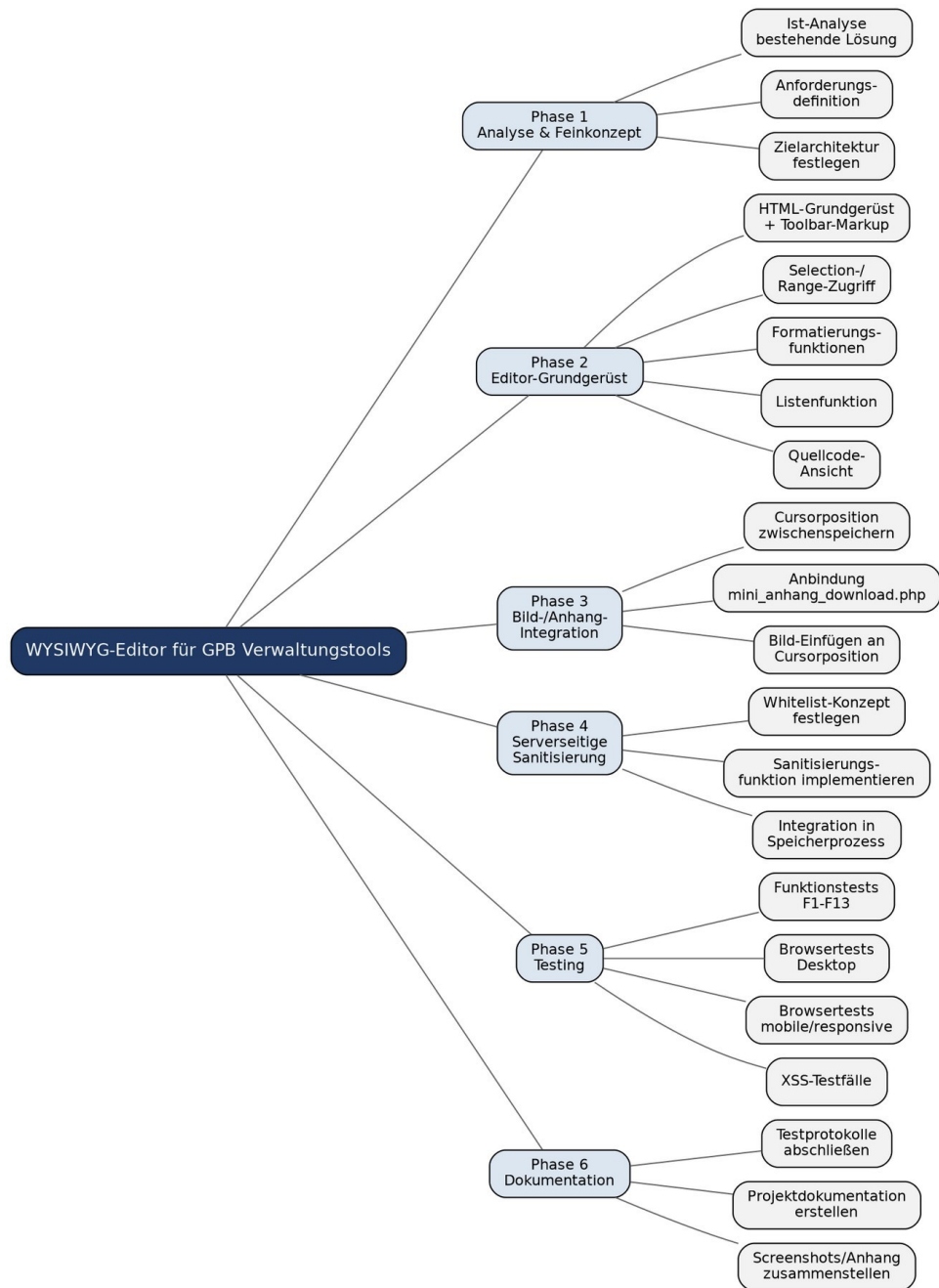


Abbildung 6: Projektstrukturplan

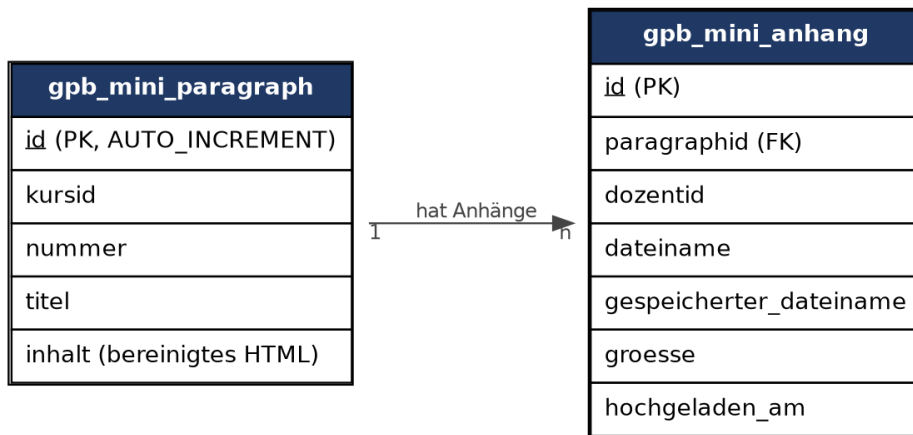


Abbildung 7: ERD

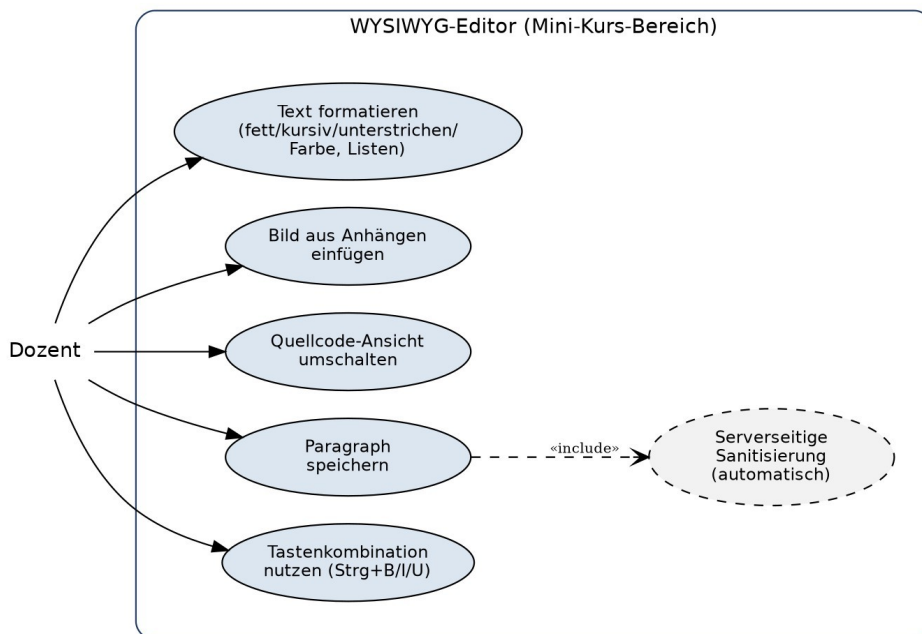


Abbildung 8: Use-Case-Diagramm

Phase	Tag 1	Tag 2	Tag 3	Tag 4	Tag 5	Tag 6	Tag 7	Tag 8	Tag 9	Tag 10
Phase 1	X									
Phase 2		X	X	X						
Phase 3					X	X				
Phase 4							X	X		
Phase 5									X	X
Phase 6									X	X

Tabelle 9: Gantt-Diagramm (Phasen je Arbeitstag)

7 Quellen

- MDN Web Docs: Selection, <https://developer.mozilla.org/de/docs/Web/API/Selection>, Stand: September 2026
- MDN Web Docs: Range, <https://developer.mozilla.org/de/docs/Web/API/Range>, Stand: September 2026
- MDN Web Docs: HTMLElement.contentEditable, <https://developer.mozilla.org/de/docs/Web/API/HTMLElement/contentEditable>, Stand: September 2026
- MDN Web Docs: Document.execCommand() (deprecated), <https://developer.mozilla.org/de/docs/Web/API/Document/execCommand>, Stand: September 2026
- PHP Manual: DOMDocument, <https://www.php.net/manual/de/class.domdocument.php>, Stand: September 2026
- PHP Manual: libxml_use_internal_errors, <https://www.php.net/manual/de/function.libxml-use-internal-errors.php>, Stand: September 2026
- WHATWG HTML Living Standard: contenteditable-Attribut, <https://html.spec.whatwg.org/multipage/interaction.html#attr-contenteditable>, Stand: September 2026